

Handling Date & Time Variables

Module 34 Study Notes • Feature Engineering

1. The Hidden Wealth in Dates & Times

In raw datasets, dates are often provided as simple strings (e.g., `"2023-08-15 14:30:00"`). A machine learning model cannot natively understand this string. However, this single string is a goldmine of hidden contextual information.

From one timestamp, you can extract:

- Is it a weekend or a weekday? (Crucial for e-commerce or traffic prediction).
- Which quarter or semester of the year is it? (Crucial for financial forecasting).
- Was the action performed in the morning, afternoon, or late at night?
- How much time has elapsed between today and the date of the event?

2. Step 1: Converting to Datetime Objects

When you load a CSV using Pandas, Date/Time columns are imported as `object` (string) types by default. You cannot extract features from a string. You must explicitly convert it into a Pandas `datetime64` object.

```
import pandas as pd

# Convert string object to datetime
df['date_column'] = pd.to_datetime(df['date_column'])
```

3. Extracting Date-Level Features

Once the column is a datetime object, you can access its attributes using the `.dt` accessor.

```
# Basic Extraction
df['year'] = df['date_column'].dt.year
df['month'] = df['date_column'].dt.month
df['day'] = df['date_column'].dt.day

# Extracting String Names
```

```
df['month_name'] = df['date_column'].dt.month_name()
df['day_name'] = df['date_column'].dt.day_name()

# Extracting Business/Academic Features
df['quarter'] = df['date_column'].dt.quarter
df['week_of_year'] = df['date_column'].dt.isocalendar().week

# Creating Custom Flags (e.g., Is it a weekend?)
import numpy as np
# In Pandas, Monday=0 and Sunday=6. So weekend is > 4.
df['is_weekend'] = np.where(df['date_column'].dt.dayofweek > 4, 1, 0)
```

4. Extracting Time-Level Features

Just like dates, times can be fractured into highly specific components.

```
# Extracting exact time segments
df['hour'] = df['date_column'].dt.hour
df['minute'] = df['date_column'].dt.minute
df['second'] = df['date_column'].dt.second

# Isolating just the time (removes the YYYY-MM-DD)
df['pure_time'] = df['date_column'].dt.time
```

5. Calculating Time Differences (Deltas)

Often, the absolute date doesn't matter as much as **how long ago** something happened (e.g., "Days since last purchase" or "Customer account age").

```
import datetime

# Get current date/time
today = datetime.datetime.today()

# Calculate the raw difference
df['time_elapsed'] = today - df['date_column']

# Extract just the pure 'Days' difference
df['days_elapsed'] = df['time_elapsed'].dt.days
```

```
# Extracting months or hours requires numpy timedeltas
# Extracting months passed:
df['months_passed'] = df['time_elapsed'] / np.timedelta64(1, 'M')

# Extracting hours passed:
df['hours_passed'] = df['time_elapsed'] / np.timedelta64(1, 'h')
```

6. Summary

Date and Time variables should almost never be passed into a model directly or dropped. By breaking down a single datetime column into 5 to 10 distinct integer columns (Year, Month, Hour, Is_Weekend, Days_Since), you provide your model with deep contextual understanding that drastically improves predictive accuracy.